

UPM Platform

The Grand Tour ■■

or: How a single number escapes a faraway database and grows up to be a chart

You said "take the risk and kick it off." I did. Now here's the museum tour of what got built — no jargon left unexplained, every tool introduced like a character with a job, and hopefully zero yawns. Grab a coffee. ■

1. The one-sentence mission (the whole point)

Telecom engineers have KPI data trapped in a faraway Oracle database across a flaky network. We want them to build dashboards on that data without hammering Oracle, and keep working even when Oracle is unreachable.

Everything below is in service of that sentence. If a design choice doesn't serve it, it got cut. The trick the whole system is built around:

Copy the data once, locally, into a fast store. Then never bother Oracle again on the read side.

That's it. That's the magic. The rest is plumbing — but it's *good* plumbing, and plumbing is where projects live or die.

2. The cast of characters ■

Think of the platform as a small town with very specialized residents.

■ The faraway warehouse	Oracle (SON)	Where the raw KPI data actually lives. Across a rickety bridge (the WAN). We visit as rarely as possible.
■ The delivery drivers	Celery workers	Drive to the warehouse, grab a box of fresh data, drop it on the loading dock.
■ The loading dock	Parquet zone landing	Where boxes of data sit after delivery. Doubles as a <u>receipt archive</u> .

■■■ The librarian	DuckDB Gateway	The <u>only</u> person allowed to write in the pantry ledger. Takes boxes, shelves them.
■ The pantry	DuckDB	The fast local store dashboards actually read from.
■■ The filing cabinet	PostgreSQL	Who's who, which jobs exist, which dashboards exist. <u>Not</u> the data itself.
■ The pneumatic tubes	Redis	Little message capsules whoosh between residents.
■ The alarm clock	Celery Beat + RedBeat	Rings every hour and tells the drivers "time to fetch fresh data."
■■ The front desk	FastAPI backend	Everyone talks to the front desk. Checks your keycard, fetches what you asked for.
■■ The showroom	React frontend	The pretty windows where viewers see charts. Talks only to the front desk.

■■■ Exactly one librarian writes to the pantry. Everyone else hands boxes to the dock and asks the librarian nicely.

Why so strict? Because DuckDB (the pantry) is a single file, and a single file can only have **one writer at a time** without corruption. So instead of *hoping* nobody else writes, we made it *structurally impossible*: only the Gateway holds the pen. This is the spine of the whole design.

3. The two great journeys ■■

Everything that happens is one of two trips.

Journey A — "Get the data in" (the Write Path) 🚚■

■ alarm rings → ■ driver visits ■ Oracle → ■ drops a Parquet box on the dock → ■ "box ready!" capsule to Redis → ■■■ librarian shelves it in ■ DuckDB → ■■ updates the catalog: "table is fresh, version 7, 588 rows"

A few clever details that make this robust instead of fragile:

- **Watermarks.** The driver doesn't re-fetch *everything* each hour. It remembers "last time I got data up to 3:00 PM" and asks Oracle for "only stuff newer than 3:00 PM." Less network, less load.
- **Idempotency** (fancy word, simple idea). If the same box gets shelved twice (because something hiccupped and retried), you don't get double the data. *Doing it twice = doing it once.* Peace of mind.
- The **dock is also a backup.** Those Parquet boxes stick around for 30 days. If the pantry burns down (file corrupts), we rebuild it from the boxes — **without ever calling Oracle.**

Journey B — "Show the data" (the Read Path) ■

■■ viewer opens a dashboard → ■■ front desk checks the keycard → asks ■■■ librarian (read-only) → ■ DuckDB runs a fast query → front desk stamps it "data as of 3:00 PM" → ■ chart appears

Notice what's not in Journey B: **Oracle.** The read path *physically cannot* reach the warehouse. Even if the bridge is on fire, dashboards keep working off the local pantry — they just show an honest **amber "stale" badge** so nobody makes a decision on old milk. ■

4. The official blueprint, decoded ■■

Here's the canonical architecture. After it, we decode **every box and every arrow** in plain words.

Neighborhoods (subgraphs):

1. **Remote Oracle (SON)** — The warehouse, across the WAN (intermittent)
2. **Write Path** — Scheduler, Ingestion Workers, Parquet Landing Zone
3. **Analytics Plane** — DuckDB Gateway (sole RW) and DuckDB store
4. **Read Path** — Backend API and React Frontend
5. **Control Plane** — PostgreSQL (metadata) and Redis (broker + cache)
6. **LLM** — On-prem Qwen3-class LLM (OpenAI-compatible)

Arrow meanings:

- (solid) = real data/commands move this way
- (dashed) = just peeking or registering metadata
- ↔ (double) = two-way conversation

5. The technology choices (and why)

Tools were chosen because each solved a specific, named problem.

The Python side (the plumbing)

- **Airflow + SQLAlchemy + Alembic** (control plane) — Why SQLAlchemy: talk to Postgres in Python objects instead of raw SQL strings. Why Alembic: database schema changes are recorded as numbered migrations.
- **DuckDB** (dataplane) — Fast analytics database in a single file, needs zero servers. Perfect for on-premises setup. Its one quirk (single writer) is why the Gateway exists.
- **Parquet** (landing zone) — Compact columnar file format DuckDB loves. Self-describing, great for long-term backup/replay.
- **FastAPI + uvicorn** (backend) — Modern, fast. Turns Pydantic contracts into documented HTTP API almost for free.
- **PyJWT + bcrypt** (auth) — Never store real passwords, store one-way hashes. JWT = tamper-proof token after login.
- **Celery + Redis + celery-redbeat** — Run background work with automatic retries. RedBeat stores schedules as data, not code.
- **python-oracledb** — Official driver with timeouts and row caps so runaway queries can't hurt anyone.

The JavaScript side (the showroom)

- **React + Vite** — UI framework and lightning-fast dev server.
- **MUI (Material UI) v6** — Polished buttons/cards/tables in Google's Material style.
- **Recharts** — Turns rows of numbers into line/bar/pie charts with a few lines of code.
- **axios** — Little courier that carries requests from showroom to front desk (clips your keycard on automatically).
- **react-router** — Single-page app has "pages" (Dashboards, Catalog, Jobs) without full reloads.

The "keep it running & honest" side

- **Docker Compose + Traefik + nginx** — docker compose up boots the entire town on any machine, identically.
- **Prometheus + Grafana** — Dashboards *about the system itself* (health, speed). Wired in, fleshed out later.
- **pytest + ruff** — 12 automated tests prove the pipeline works; ruff keeps code tidy. Both run automatically in CI.

6. The cleverest decisions (the "why is it like *that*?" answers) ■

■ The single-writer Gateway

DuckDB allows one writer. Two processes writing = corruption. So rather than coordinate carefully, we made writing a **funnel**: workers never open DuckDB; they drop Parquet boxes and send a "load this" message. The **one** Gateway (living inside the backend) consumes those messages and writes, one at a time. *Discipline fails at 3 AM; architecture doesn't.*

■ The synthetic data source (the body double) ■

There's no live Oracle warehouse attached. Rather than leave the pipeline untested, the ingestion source is **pluggable**, and the default is a **synthetic generator** that invents realistic telecom KPIs. So the entire pipeline runs and is provable **today**, with no Oracle. Flip UPM_SOURCE_TYPE=oracle + creds, and the real driver takes over — same plumbing, real data.

■ Two modes from one codebase: "dev" and "prod"

Dev mode collapses the town into **one process** using SQLite (local filing cabinet) and in-process librarian. `uv run upm demo` and you're live in seconds — no Docker, no servers. Prod mode has Redis + Postgres + background workers. *Same code*, just more residents. This is why it runs on a laptop and it's ready for a real server.

■ Freshness is a first-class citizen

Stale data in telecom = bad decisions. Every table carries "last successfully loaded at", every query response carries "data as of", and the UI shows a **green badge** (fresh) or **amber badge** (stale) on each widget. *Honesty over optimism.* The internal "version number" also auto-invalidates the cache.

7. Follow one number all the way ■ (the payoff)

Let's trace a single `traffic_eri` value from birth to chart:

■ You run upm demo (or the hourly alarm fires). → apps/scheduler says "run job load_hybrid_cs_cell."

■ apps/ingestion/orchestrator.py asks the source for data. The synthetic source (sources/synthetic.py) invents `traffic_eri = 42.3` for cell `CELL_0001`, region North, 3 PM. → Writes Parquet to `data/landing/hybrid_cs_cell/12.parquet`.

■ It hands a `LoadCommand` to the librarian (in dev, directly; in prod, via Redis).

■■■ packages/dataplane/gateway.py shelves the box into DuckDB (staging → atomic swap), then writes to filing cabinet: "hybrid_cs_cell is version 1, 588 rows, fresh at 3 PM."

■ A viewer opens **Hybrid Cell Overview**. Frontend POSTs a structured query to `/api/query`.

■■ apps/backend/routers/query.py checks the keycard, asks sql-tools to safely build `SELECT` query, runs it read-only through the Gateway.

■ Back comes data, stamped "data as of 3:00 PM", and Recharts draws the bar. Your 42.3 is now a pixel. ■

That whole trip — and the RBAC, freshness, and cache-hit behavior around it — is exactly what the 12 automated tests verify.

8. What's real vs. what's a placeholder (no fibbing) ■

Built & tested ■

- ✓ Login, roles/keycards (Admin/Builder/Viewer), per-project access
- ✓ Data catalog with freshness badges
- ✓ Dashboards: line/bar/area/pie/KPI/table charts
- ✓ Full write pipeline (synthetic) with full/append/upsert + idempotency
- ✓ Job runner + run history, "Run now" button
- ✓ Query cache, freshness/staleness, audit log

Scaffolded, finished later ■

- In-app dashboard editor UI (dashboards are API-editable now)
- Map widgets (MapLibre) on geo data
- Retention/compaction cleanup jobs
- Live Oracle preview/EXPLAIN (driver ready, no server yet)
- AI chat tool-loop (safety guard ships now; loop awaits LLM endpoint)
- Prometheus app metrics export

9. The TL;DR for your future self ■

- **One idea:** copy Oracle's data locally once; read from the copy forever.
- **One rule:** only the Gateway writes to DuckDB. Everyone else hands it a box.
- **One safety net:** the boxes (Parquet) are also the backup — rebuild without Oracle.
- **One promise to users:** if data is old, we show it (amber badge), we don't hide it.
- **One codebase, two scales:** runs on your laptop in one process, or as a real distributed town.
- **Everything is data, not code:** new feeds and dashboards are rows in a database, not deploys.

That's the whole town. Tools were chosen because each solved a specific, named problem — not for résumé points. And the body-double synthetic source means you can watch the entire machine run **right now**, before the real Oracle ever picks up the phone.

Tour over. Mind the gap, and please don't feed the librarian. ■■■■■

Want the formal version? See the architecture plan, ADR directory, and README for details on how to run it.